UNIVERSIDADE FEDERAL DE ALAGOAS

Campus A. C. Simões

Engenharia da Computação

EMILY VITÓRIA DE JESUS ROCHA

Linha de Produto de Software para Comércio Eletrônico

Modelagem e implementação das variantes de roupas e alimentos

Tópico em Engenharia de Software

Projetando Linhas de Produto de Software

Relatório do projeto | Semestre 2026.2

Perfil GitHub informado pela autora

https://github.com/emilyrch

Endereço específico do repositório ainda não informado

Maceió

2026

Resumo

Este relatório apresenta a modelagem e a implementação de uma Linha de Produto de Software (LPS) para comércio eletrônico, derivada em duas variantes: uma loja de roupas com combinações explícitas de tamanho e cor e uma loja de alimentos embalados, com informações alimentares e entrega agendada. A solução compartilha o núcleo de catálogo, carrinho, revisão da compra, pedidos e pagamento Pix inteiramente simulado. A variabilidade é declarada em manifestos estritos, interpretados na inicialização, e vinculada a componentes de domínio selecionados no ponto de composição.

A arquitetura compreende um frontend Next.js, que também atua como intermediário de APIs, e quatro aplicações FastAPI independentes: catálogo, pedidos, pagamentos e recomendações. Catálogos fictícios são validados em memória; pedidos, pagamentos e recomendações utilizam bancos SQLite privados. A criação de pedidos emprega revisões persistidas, chaves de idempotência, restrições de unicidade e recuperação limitada de estados intermediários, sem transação distribuída.

O registro histórico anterior contém 67 testes completos de backend. No incremento atual, passaram 43 testes offline, 16 testes web e 24 vetores de configuração; esses resultados não substituem testes HTTP completos. Na verificação posterior, lint e typecheck passaram, mas o build do frontend foi impedido por acesso negado no compilador SWC. Posteriormente, uma execução manual da variante de roupas pelo navegador concluiu revisão, pagamento simulado e consulta do pedido, com capturas reais. Isso não homologa alimentos, reinício, build de produção, toda a suíte E2E ou containers. O microsserviço de recomendação por preferências e histórico foi acrescentado, com imagens locais de dez peças, seletor visual da LPS e identificação dos serviços na interface.

Palavras-chave: linha de produto de software; reuso; variabilidade; microsserviços; componentes; comércio eletrônico.

Delimitação e identificação documental

O texto segue os itens do documento “Elementos do Relatório do Projeto -2026-2”, entregue pela autora. A análise considera o código e os registros locais disponíveis em 10 de novembro de 2026. Nomes de cerimônias, durações de sprint, resultados de telas e publicação em GitHub não foram presumidos. Os backlogs são uma organização retrospectiva dos incrementos registrados.

Nota de evidência. O brasão oficial da UFAL não foi incorporado: o acesso ao arquivo institucional foi bloqueado no ambiente de produção do relatório. O nome do docente e o URL específico do repositório não foram informados. Não se utilizou um logotipo redesenhado ou uma URL de projeto inventada.

Sumário

1 Enunciado do projeto 4

2 Metodologia ágil escolhida 4

3 Técnicas de reuso de software utilizadas 5

4 Desenvolvimento segundo a metodologia e o reuso 5

5 Product Backlog e Sprint Backlogs 6

5.1 Diagrama de features 9

5.2 Arquitetura em camadas e variabilidade 10

5.3 Componentes de software e variabilidade 11

5.4 Configuração e construção de produtos 13

6 Arquitetura baseada em microsserviços 15

6.1 Recomendação considerando o histórico 15

6.2 Especificação documentada do API Gateway 16

6.3 Especificação dos microsserviços 17

6.4 Portas e evidência dos serviços em execução 21

6.5 Construção de um produto a partir da LPS 23

7 Tecnologias utilizadas na implementação 24

8 Funcionamento do produto e telas 25

9 Código do projeto e disponibilização no GitHub 35

10 Referências bibliográficas 36

1 Enunciado do projeto

O projeto consiste em desenvolver uma família de aplicações de comércio eletrônico com ativos compartilhados e variabilidade explícita. Em vez de manter duas aplicações copiadas, a LPS deve derivar produtos a partir do mesmo código e de uma configuração válida. Roupas e alimentos diferem nas regras de seleção e de entrega, preservando a experiência básica de compra e os serviços comuns.

As exigências inicialmente fornecidas foram a modelagem e a implementação de uma LPS, o uso de microsserviços e o uso de componentes de software. Python foi sugerido e agentes de inteligência artificial foram considerados opcionais. As escolhas de Next.js, FastAPI, SQLite, inicialmente três serviços e demonstração local foram decisões de projeto. A leitura posterior do roteiro de relatório acrescentou evidências e um requisito funcional de recomendação que não podem ser considerados atendidos apenas pelo escopo anterior.

Produto	Variabilidade obrigatória	Núcleo reutilizado
Roupas — essencial	Combinações válidas de tamanho/cor; sem agendamento ou promessa de prazo	Catálogo, carrinho, endereço, revisão, pedidos, Pix simulado e consulta
Alimentos — despensa	Ingredientes, alergênicos, nutrição por porção; janela futura obrigatória	Mesmo frontend e serviços, com políticas selecionadas pelo manifesto

Cada implantação representa uma loja, com dados e segredos separados. A demonstração sequencial das variantes é suficiente para o escopo adotado. Não há estoque, reserva física, login, cartão, desconto, capacidade de agenda ou transferência financeira real. Produtos e informações alimentares são exclusivamente fictícios, sem alegação de certificação regulatória.

2 Metodologia ágil escolhida

A organização observada é iterativa e incremental, orientada por backlog e revisão de entregas. Cada incremento delimita um objetivo verificável, depende dos anteriores e registra decisões e limitações. O relatório utiliza práticas de priorização e inspeção de incrementos associadas ao desenvolvimento ágil (Beck et al., 2001), sem afirmar a realização de eventos ou papéis para os quais não há evidência.

Para apresentar o Product Backlog e o Sprint Backlog pedidos no roteiro, os incrementos foram agrupados retrospectivamente em sete sprints documentais. Não há registro de timeboxes, Daily Scrum, Product Owner ou Scrum Master; portanto, não se declara conformidade integral ao Scrum definido por Schwaber e Sutherland (2020).

3 Técnicas de reuso de software utilizadas

A LPS concentra o reuso no domínio do comércio eletrônico: as duas variantes compartilham contratos, infraestrutura de configuração, interface, cálculo em centavos e fluxo de compra. A derivação ocorre por ativos comuns e regras de variabilidade, coerentemente com o desenvolvimento a partir de ativos compartilhados descrito por Northrop et al. (2012). O domínio é uma alternativa exclusiva; não existem features opcionais desligáveis no MVP.

Técnica	Aplicação concreta	Benefício e limite
Linha de Produto de Software	Manifestos clothing/food, modelo de features, schemas e composição de domínio	Uma base para dois produtos; novas variantes exigem regras e configuração explícitas
Microsserviços	Catálogo, pedidos, pagamentos e recomendações com processos, APIs e armazenamento privados	Responsabilidades independentes; comunicação sujeita a falhas e timeouts
Componentes de domínio	CatalogLoader, ProductSelectionValidator, DeliveryPolicy, TotalsCalculator e gateway de pagamento	Substituição por contrato; seleção concentrada no bootstrap
Componentes visuais	Storefront, ProductMedia e FoodInformation	Interface comum com extensões do domínio; não são microsserviços

As camadas compartilhadas não permitem que um serviço abra o banco privado de outro. Pedidos resolve seleções por HTTP no catálogo e observa/autoriza pagamentos pela API interna. Compartilhar tipos e bibliotecas não significa compartilhar propriedade dos dados. O catálogo não consulta banco de pedidos, e o navegador não acessa diretamente a resolução interna ou a autorização de pagamentos.

4 Desenvolvimento segundo a metodologia e o reuso

O desenvolvimento partiu do diagnóstico do protótipo Básica/Premium, preservado como histórico. Em seguida, foram definidos o núcleo, o domínio exclusivo, os contratos e os componentes. A fundação materializou manifestos, processos, persistência e ambiente; os incrementos seguintes implementaram catálogo, revisão, admissão e pagamento recuperável. Por último, o frontend foi integrado no código e submetido à estabilização possível no ambiente.

A inspeção de cada incremento orientou o próximo: validações de identidade precederam o catálogo; catálogos e políticas de entrega precederam revisões; revisões duráveis precederam pedidos; a admissão local precedeu a autorização simulada. Essa ordem reduziu o acoplamento entre evolução visual e regras de compra. A documentação diferencia artefato implementado, teste executado e evidência ainda pendente.

5 Product Backlog e Sprint Backlogs

O Product Backlog abaixo foi reconstruído a partir do escopo e dos arquivos do projeto. A prioridade representa dependência técnica e importância para a demonstração, não uma estimativa histórica de esforço. “Implementado” refere-se ao código; a homologação é informada separadamente.

ID	Item de backlog e aceitação resumida	Situação
PB01	Modelar núcleo, domínio XOR, dependências e configurações válidas	Documentado e implementado
PB02	Validar manifestos e produzir identidade/hash iguais em Python/TypeScript	Implementado; vetores aprovados
PB03	Iniciar os serviços com saúde, credenciais e dados privados	Implementado; suíte backend aprovada
PB04	Disponibilizar catálogo estrito: dez roupas e seis alimentos	Implementado; testado
PB05	Validar combinações de roupa e metadados alimentares	Implementado; testado
PB06	Gerar janelas futuras de alimentos e rejeitar agenda em roupas	Implementado; testado
PB07	Persistir revisão imutável por dez minutos, com total do servidor	Implementado; testado
PB08	Admitir pedidos idempotentes e consumir cada revisão uma única vez	Implementado; testado
PB09	Autorizar Pix simulado durável e recuperar incertezas sem duplicação	Implementado; suíte aprovada
PB10	Oferecer sessão anônima, intermediário e carrinho por loja	Implementado; testes web aprovados
PB11	Exibir catálogo, checkout e consulta nas duas identidades visuais	Implementado; validação visual pendente
PB12	Validar build, fluxo Next→backend, instalação limpa e Compose	Parcial; bloqueios do ambiente
PB13	Atender recomendação por histórico em serviço próprio — item 6.1 do roteiro	Implementado; testes offline e de fronteira aprovados; HTTP atual pendente
PB14	Entregar seletor visual, capturas de telas/portas e URL do projeto	Seletor visual implementado; capturas e URL pendentes

Nota de evidência. PB13 e parte de PB14 não integravam o escopo inicialmente aprovado. A recomendação e o seletor visual foram implementados em incremento posterior. Capturas reais e URL específico continuam pendentes; código não substitui homologação.

5 Continuação dos Sprint Backlogs

Cada sprint documental reúne objetivo, seleção do backlog, tarefas e evidência. A numeração descreve a sequência dos incrementos; não se atribuem datas, reuniões ou duração inexistentes.

Sprint 1 — Diagnóstico e modelagem

Objetivo: Definir a família e preservar o protótipo. **Backlog:** PB01.

Tarefas: Analisar estrutura/dependências; distinguir exigências e decisões; definir features, contratos, componentes e limites.

Incremento e evidência: docs/diagnostico.md, modelo-lps.md e arquitetura-atual.md; histórico Básica/Premium separado.

Sprint 2 — Fundação executável

Objetivo: Derivar configurações e iniciar a arquitetura. **Backlog:** PB02, PB03.

Tarefas: Criar manifestos estritos; hash comum; três apps FastAPI; schema privado; saúde e lançador; preparar Compose.

Incremento e evidência: foundation/, services/*/app.py, configurations/, tools/lps.py e testes de fundação.

Sprint 3 — Catálogo e domínio

Objetivo: Oferecer dados próprios e regras reutilizáveis. **Backlog:** PB04, PB05, PB06.

Tarefas: Carregar catálogos imutáveis; validar seleções; implementar totais e políticas de entrega com relógio injetável.

Incremento e evidência: demo-data/, domain/, services/catalog/ e testes de catálogo; APIs internas implementadas.

Sprint 4 — Revisão persistida

Objetivo: Preparar a confirmação sem criar pedido prematuramente. **Backlog:** PB07.

Tarefas: Resolver produtos por HTTP; validar endereço/janela; calcular total; persistir snapshot e TTL de dez minutos.

Incremento e evidência: services/orders/service.py, repository.py e rotas de revisão; cenários positivos/negativos testados.

5 Conclusão dos Sprint Backlogs

Sprint 5 — Pedido e pagamento recuperável

Objetivo: Admitir e confirmar compras com persistência. **Backlog:** PB08, PB09.

Tarefas: Reservar chave; revalidar revisão; admitir em transação curta; observar/autorizar Pix; recuperar com leases e tentativas limitadas.

Incremento e evidência: services/orders/checkout.py, order_repository.py, worker.py e services/payments/. A suíte completa registrada aprovou 67 testes de backend.

Sprint 6 — Interface compartilhada

Objetivo: Expor o fluxo das duas variantes sem divulgar credenciais. **Backlog:** PB10, PB11.

Tarefas: Construir identidade visual, catálogo, detalhe, carrinho, checkout e consulta; implementar cookie assinado, contexto e gateway por rotas fixas.

Incremento e evidência: apps/store-ropa/src/. Registro anterior: dez testes da camada web aprovados; não comprovam renderização ou E2E.

Sprint 7 — Estabilização e evidências

Objetivo: Conferir implementação e tornar bloqueios explícitos. **Backlog:** PB12 e levantamento de PB13/PB14.

Tarefas: Corrigir reconciliação do carrinho e validação de respostas; executar verificações; preparar E2E isolado; registrar limitações e roteiro.

Incremento e evidência: tools/validate.py, tools/e2e.py e docs/homologacao-2026-11-10.md. Lint/typecheck passaram em checagem posterior; build falhou por acesso negado no SWC.

Critérios de conclusão e tratamento das pendências

Uma tarefa não é considerada homologada apenas porque o código existe. Para regras de domínio, exige-se validação de cenários pertinentes; para a aplicação completa, exige-se build, execução da interface e fluxo entre Next.js e serviços. Na estabilização, bloqueios externos foram mantidos no relatório, em vez de substituir ferramentas reais por simulações ou alterar pacotes globais.

O incremento adicional implementou recomendações por preferências e histórico, dez imagens locais, seletor visual em /configurador e painel de microsserviços. Posteriormente, foram coletadas capturas de roupas e observada uma compra confirmada. A homologação completa de ambas as variantes permanece pendente. O projeto não é uma LPS de jogos; o vídeo específico pedido no item 11 não se aplica.

5.1 Diagrama de features

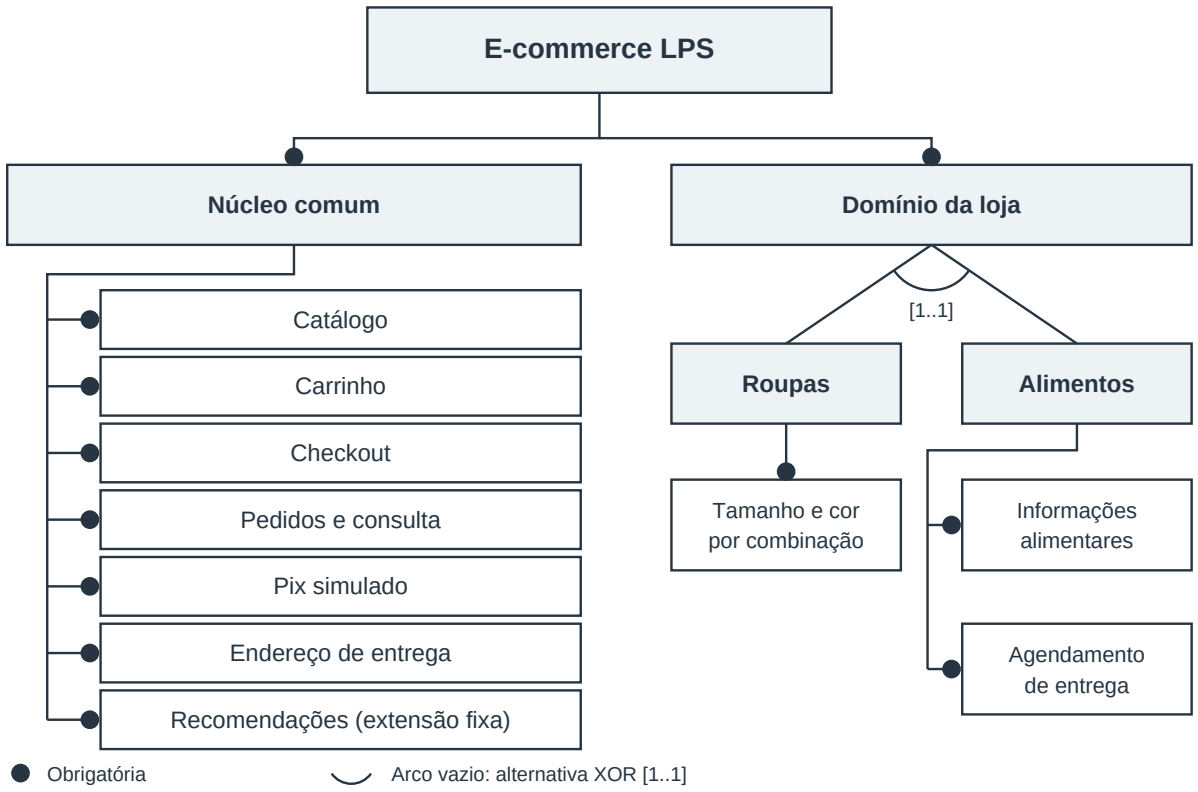


Figura 1 — Modelo de features com obrigatoriedade e grupo alternativo XOR [1..1]. Notação de features, distinta da UML. Fonte: elaboração própria a partir de docs/modelo-lps.md.

O núcleo é obrigatório nos dois produtos. O grupo de domínio tem cardinalidade 1..1: roupas ou alimentos. Seleção de tamanho/cor exige roupas; informação alimentar e agendamento exigem alimentos e são inseparáveis nesta versão. Não se classifica como opcional uma capacidade que o projeto não permite desligar.

Feature	Roupas	Alimentos
Catálogo, carrinho, checkout e pedidos	Obrigatória	Obrigatória
Endereço e Pix simulado	Obrigatória	Obrigatória
size_color_selection	Obrigatória	Excluída
food_information	Excluída	Obrigatória
delivery_scheduling	Excluída	Obrigatória
Features opcionais desligáveis	Nenhuma	Nenhuma

Seja K o conjunto {catalog, cart, checkout, orders, simulated_pix, address_delivery}. As configurações válidas são K acrescido de size_color_selection para roupas, ou K acrescido de food_information e delivery_scheduling para alimentos. Exemplos inválidos incluem features duplicadas, domínio food com size_color_selection, roupas com entrega scheduled ou ausência de componente obrigatório.

5.2 Arquitetura em camadas e variabilidade

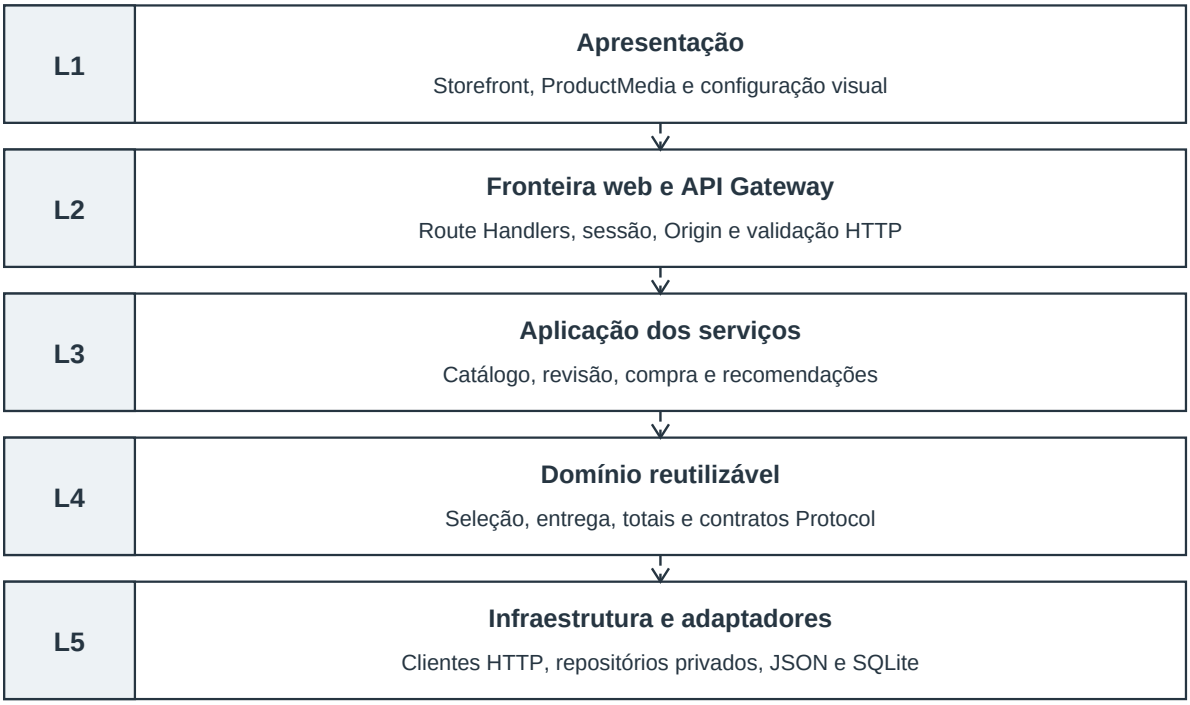
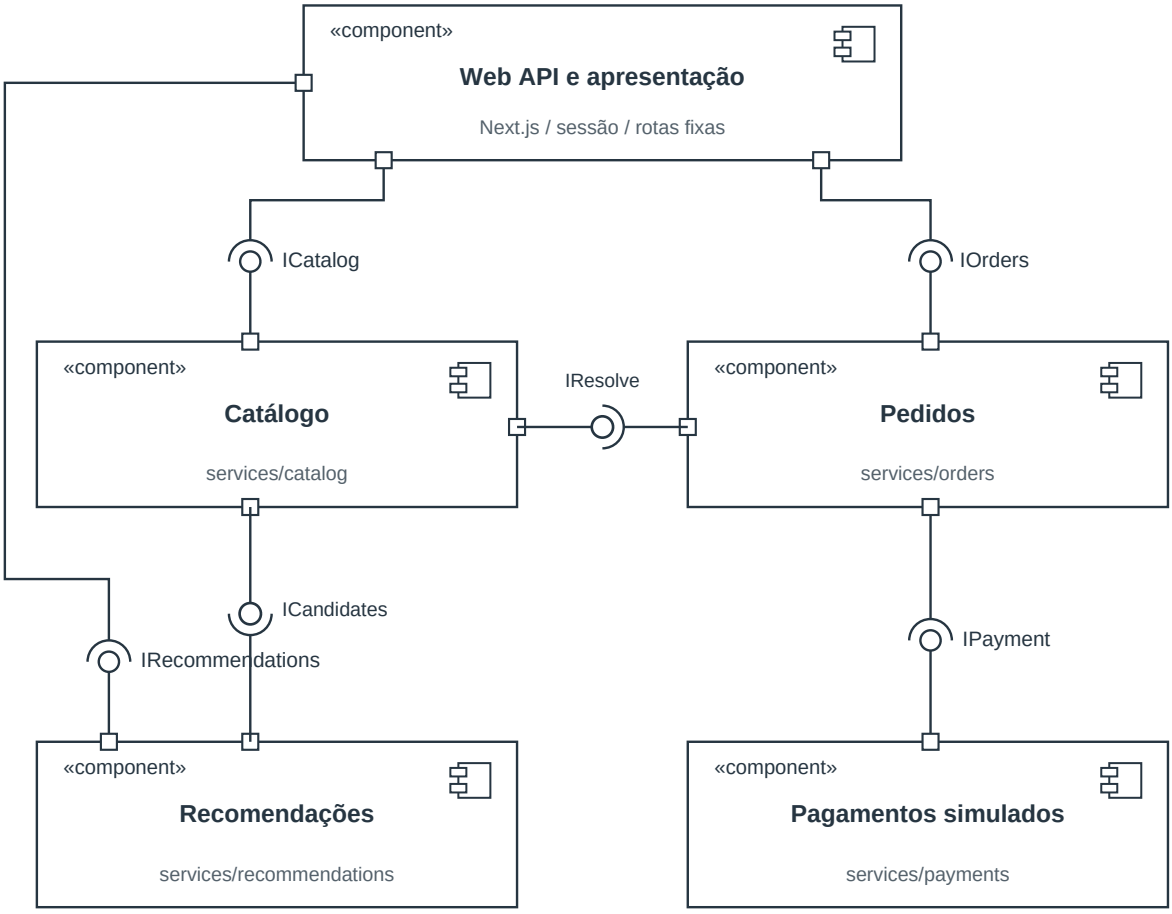


Figura 2 — Visão lógica em camadas e direção de dependência. Não representa sequência temporal nem processos distintos. No MVP, PaymentsRepository reúne autorização simulada e adaptação ao SQLite.

A apresentação coleta escolhas, mas não define o preço final. O intermediário autentica a sessão e recompõe cabeçalhos internos. A aplicação coordena casos de uso; o domínio valida seleções, entrega e totais; a infraestrutura executa I/O. O diagrama de camadas complementa a implantação em microsserviços: não representa um único banco ou um processo monolítico.

Ponto de variação	Variante roupas	Variante alimentos	Vinculação
Domínio da loja	clothing	food	Bootstrap pelo manifesto
Seleção	ClothingSelectionValidator	FoodSelectionValidator	Registro em compose
Entrega	UnscheduledDelivery	ScheduledDelivery	Registro em compose
Apresentação	Tamanho/cor e identidade essencial	Metadados, agenda e identidade despensa	Configuração pública
Dados de catálogo	clothing-demo-v1	food-demo-v1	Carga inicial validada

5.3 Componentes de software e variabilidade



Círculo: fornecida; semicírculo: requerida; quadrado: porta UML.

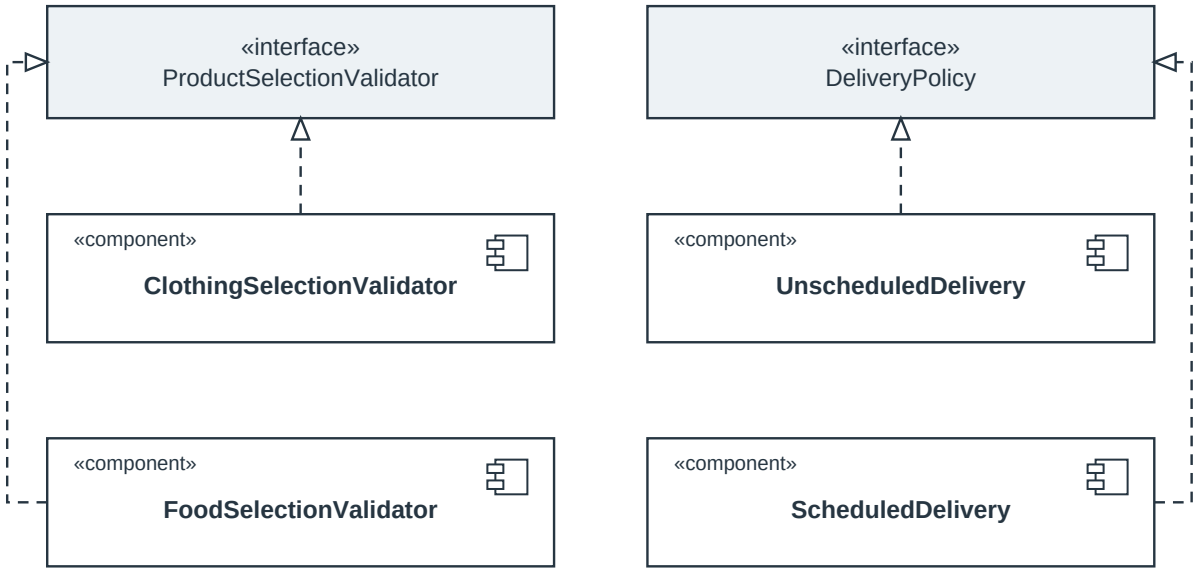
Interfaces conceituais dos contratos HTTP; porta UML não é porta TCP.

Figura 3 — Diagrama UML de componentes e interfaces HTTP da aplicação. Ícone de componente, portas e conectores de montagem com interfaces fornecidas/requeridas. Fonte: elaboração própria; convenções UML 2.5.1 (OMG, 2017).

Os nomes IPublicCatalog, IResolve, IOrders e IPayment identificam interfaces conceituais dos contratos HTTP já implementados; não representam classes ou serviços novos. Cada retângulo denota uma unidade modular. Nesta visão, as unidades de backend também correspondem aos serviços implantáveis, mas componente lógico e microsserviço não são sinônimos.

O círculo representa uma interface fornecida; o semicírculo, uma interface requerida. O conector de montagem indica que o contrato requerido é satisfeito. Quadrados na borda são portas UML de interação, sem relação com os números TCP da implantação.

5.3 Variabilidade dos componentes de domínio



{xor} por ponto de variação; seleção na inicialização pelo manifesto.
clothing: ClothingSelectionValidator + UnscheduledDelivery
food: FoodSelectionValidator + ScheduledDelivery

Figura 4 — Realização dos contratos e alternativas dos componentes de domínio. Linha tracejada e triângulo vazio indicam realização conceitual dos Protocols por tipagem estrutural, não herança explícita. A restrição {xor} seleciona uma alternativa por ponto de variação.

Contrato	Entradas e saídas	Uso e seleção
CatalogLoader	Manifesto e fonte → snapshot validado ou erro	Catálogo; JsonCatalogLoader compartilhado
ProductSelectionValidator	Produto e seleção → item resolvido ou erro	Catálogo; Clothing/Food pelo domínio
DeliveryPolicy	Endereço, slot e relógio → snapshot; list_slots → janelas	Pedidos; agendada ou não agendada
TotalsCalculator	Itens resolvidos → linhas, subtotal e total em centavos	Pedidos; cálculo puro, sem I/O
SimulatedPaymentGateway	Pedido/valor persistidos → autorização observada ou erro	Pedidos; cliente de API interna de pagamentos

Os contratos de domínio usam Protocol em domain/contracts.py. O registro em domain/composition.py seleciona estratégias no início, evitando espalhar condicionais pelos handlers. O gateway de pagamento é uma fronteira de aplicação implementada em services/orders/payment_client.py; sua autorização durável pertence ao serviço de pagamentos. Componentes visuais são unidades React e não substituem os contratos de domínio.

5.4 Configuração e construção de produtos

A configuração real é declarativa e versionada. Os arquivos configurations/clothing.json e configurations/food.json incluem schema_version, store_id, domínio, features, fonte de catálogo, entrega e pagamento. Schemas fechados rejeitam propriedades desconhecidas, features duplicadas e combinações de domínio incompatíveis. O hash SHA-256 determinístico considera a normalização documentada e é comparado nas chamadas entre processos.

Propriedade	Roupas	Alimentos
store_id	roupas-demo	alimentos-demo
domain	clothing	food
features de domínio	size_color_selection	food_information, delivery_scheduling
delivery.policy	unscheduled	scheduled
Horizonte e janelas	Não aplicável	7 dias; 09h–12h e 14h–17h
payment.provider	simulated_pix	simulated_pix
discount_cents / shipping_cents	0 / 0	0 / 0

Seleção efetivamente implementada

```
python tools/lps.py init clothing
python tools/lps.py run clothing --dev

# Após encerrar a primeira variante:
python tools/lps.py init food
python tools/lps.py run food --dev
```

O lançador seleciona o manifesto e inicia o mesmo frontend e os mesmos quatro serviços. A derivação utiliza infraestrutura, contratos e políticas comuns; não cria uma cópia independente do código. Os dados privados ficam em var/<store_id>/orders e var/<store_id>/payments. Recomendações possui seu banco em var/<store_id>/recommendations. A sacola é separada por loja no navegador. Ao reiniciar a mesma loja, devem ser preservados bancos, segredos e cookie da sessão.

A tela /configurador permite escolher exclusivamente roupas ou alimentos, visualizar as features obrigatórias, baixar o manifesto validado e obter os comandos de construção e execução. A troca de processo é explícita pelo lançador; a tela não modifica a loja ativa nem executa shell no servidor. O download corresponde ao manifesto versionado da variante. A Figura 5 registra a tela de configuração efetivamente aberta, com roupas selecionada.

Figura 5 - Configuração visual da variante de roupas

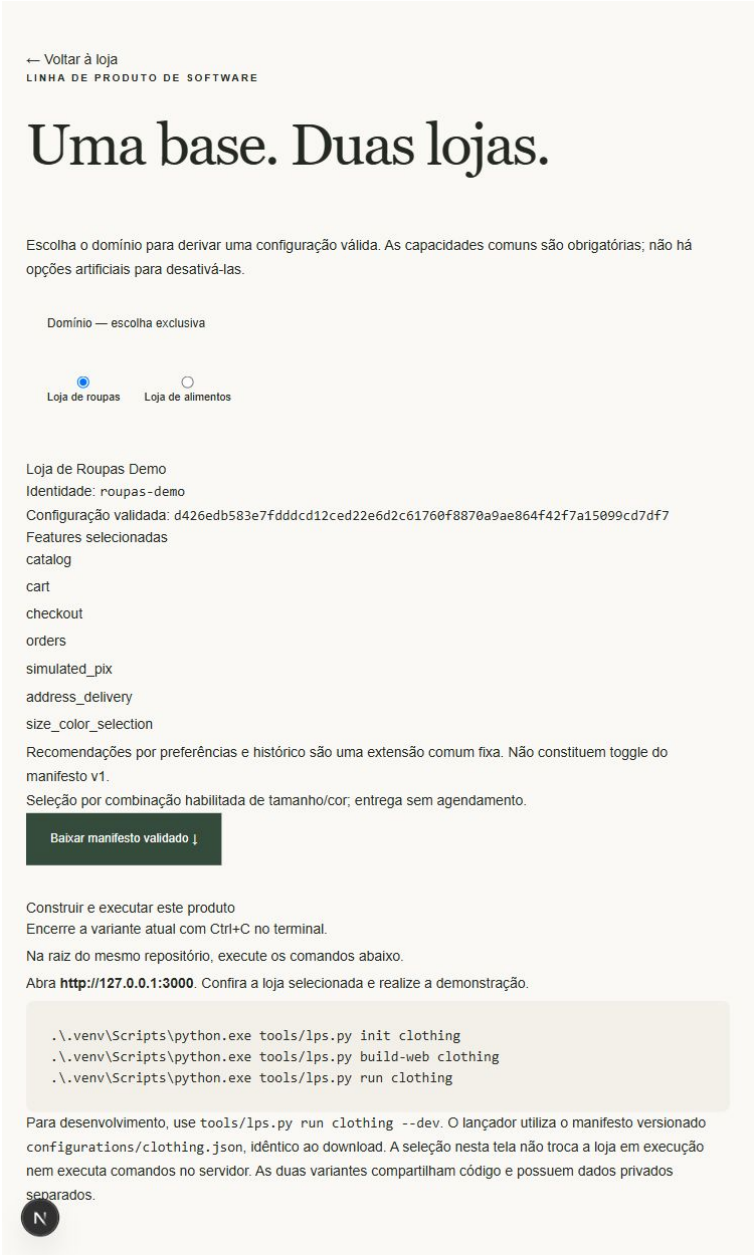


Figura 5 - Configuração visual da variante de roupas. Fonte: captura real do sistema pela autora, assistida por ferramenta de automação (10 nov. 2026).

A escolha exclusiva roupas está ativa. A tela apresenta identidade, hash, features e comandos de derivação. A presença da alternativa alimentos não representa execução ou captura dessa variante.

6 Arquitetura baseada em microsserviços

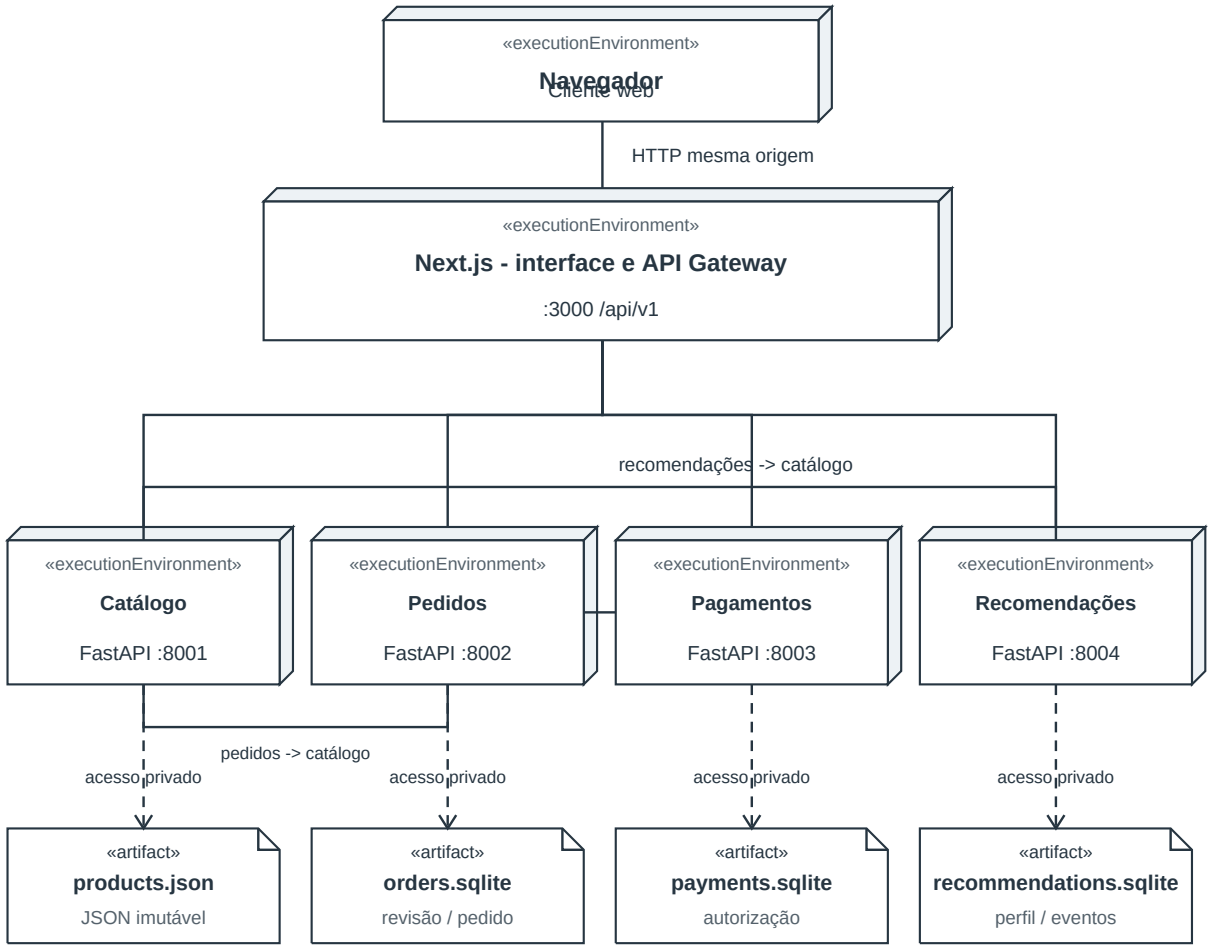


Figura 6 — Visão de implantação local com ambientes de execução, comunicação HTTP e artefatos privados. Linhas contínuas: comunicação entre processos; tracejadas: dependência de artefato. Checagens de prontidão foram omitidas para legibilidade. Diagrama arquitetural, não captura de execução.

Cada microsserviço executa em processo próprio. Pedidos consulta o catálogo por API e acessa pagamentos por credencial exclusiva. Catálogo mantém arquivos fictícios validados e congelados durante execução. SQLite é utilizado somente no serviço proprietário dos dados. O Compose prevê somente a porta 3000 publicada; no modo local, os serviços usam loopback e credenciais internas.

6.1 Recomendação considerando o histórico

O serviço services/recommendations executa em processo próprio na porta 8004. Obtém candidatos por API autenticada do catálogo, persiste preferências e até cem eventos por sessão/loja em SQLite privado e retorna até seis sugestões explicadas. Não lê bancos de outros serviços. O algoritmo é determinístico, sem IA em execução.

Categorias e histórico orientam a pontuação; orçamento limita o preço e tamanho/cor devem corresponder à mesma combinação habilitada. O usuário pode limpar preferências e pausar a coleta. Há testes de modelos, ranking, isolamento e persistência, além de contratos web; os testes HTTP novos aguardam ambiente funcional.

6.2 Especificação documentada do API Gateway

O Next.js atua como Backend for Frontend e gateway de rotas fixas. A entrada está em src/app/api/v1/[...path]/route.ts e delega à função createWebApi de src/lib/web-api.ts. O intermediário valida método, origem e JSON, aplica limites e timeouts, autentica cookie e recompõe os cabeçalhos de serviço. Route Handlers utilizam Request/Response e constituem a fronteira HTTP da aplicação (Vercel, documentação Next.js 15).

```
// src/app/api/v1/[...path]/route.ts
import { createWebApi } from '@/lib/web-api';
export const runtime='nodejs';
export const dynamic='force-dynamic';
export const GET=createWebApi();
export const POST=GET;
```

Rota pública	Destino e finalidade
POST /api/v1/session	Local; cria/reutiliza cookie assinado
GET /api/v1/store	Local; projeção pública, sem segredos
GET /api/v1/products[/{id}]	Catálogo; listagem/detalhe
GET /api/v1/delivery-slots	Pedidos; janelas de alimentos
POST /api/v1/reviews	Pedidos; revisão com sessão
POST /api/v1/orders	Pedidos; admissão com Idempotency-Key
GET /api/v1/orders/{id}	Pedidos; snapshot do dono
GET /api/v1/order-attempts/{key}	Pedidos; retomada por chave
GET/POST /api/v1/preferences	Recomendações; preferências da sessão
GET /api/v1/recommendations	Recomendações; sugestões explicadas
POST /api/v1/recommendation-events	Recomendações; histórico limitado
POST /api/v1/recommendation-reset	Recomendações; limpar e pausar

O cookie tem SID aleatório, loja, emissão/expiração e HMAC-SHA256; HttpOnly, SameSite=Lax e duração de 30 dias. Secure é usado em HTTPS. O intermediário deriva session_owner e transmite X-Session-Owner somente em chamadas privadas autenticadas. SID e credenciais não são enviados no JSON nem persistidos em localStorage.

O limite do corpo é 64 KiB; respostas internas são limitadas a 1 MiB. O timeout é de seis segundos, ou onze para admissão; o cliente usa quinze segundos. Não são seguidos redirecionamentos nem repetidas mutações automaticamente. Respostas privadas usam no-store; identidade/hash e coerência comercial são conferidos antes de responder.

6.3 Especificação dos microsserviços

Serviço de catálogo

A entrada services/catalog/app.py utiliza o bootstrap comum. services/catalog/routes.py mantém handlers pequenos e delega à aplicação CatalogService. Os modelos são estritos e discriminados por kind. A seleção de roupas exige combination_id habilitado e pertencente ao produto; alimentos proíbe esse campo, inclusive null. Não são aceitos preços do cliente. O catálogo não possui banco relacional.

Método e rota interna	Permissão	Resposta ou erro
GET /internal/v1/products	web	200; store_id, catalog_revision, items
GET /internal/v1/products/{id}	web	200 produto; 404 desconhecido
POST /internal/v1/selections/resolve	orders	Lista integral resolvida; sem resultado parcial
GET /health/live	Pública	Vida do processo
GET /health/ready	Interna	Identidade, configuração e catálogo carregado

```
# services/catalog/routes.py — excerto real
async def products(request: Request):
    return request.app.state.catalog.list_products()

async def product(product_id: str, request: Request):
    return request.app.state.catalog.detail(product_id)

async def resolve(body: ResolveRequest, request: Request):
    return request.app.state.catalog.resolve(body)
```

Listagem 1 — Delegação dos handlers. Os decorators e dependências de permissão estão no arquivo integral.

A resolução recebe entre uma e cinquenta seleções únicas, com quantidades estritamente inteiras de 1 a 100. Dois tamanhos/cores do mesmo produto podem ocupar linhas distintas; repetir a mesma identidade de seleção é inválido. O catálogo ativo contém dez roupas com imagens locais e seis alimentos, com preços em centavos e IDs estáveis.

A revisão comercial é determinística: alterações relevantes de preço, combinação ou informação alimentar mudam catalog_revision; formatação JSON e reordenação de produtos/combinações não mudam o conteúdo normalizado. Ausência ou inconsistência do catálogo impede a inicialização, sem expor caminhos privados nas respostas.

6.3 Serviço de pedidos

Pedidos coordena os casos de uso em ReviewService e Checkout. Seu banco privado conserva reviews, attempts e orders no schema 1. Revisões contêm entrada normalizada, itens e valores resolvidos, entrega, identidade/config_hash e expiração de dez minutos. A revisão não reserva estoque ou preço e não representa pedido confirmado.

Rota interna	Contrato e comportamento
GET /internal/v1/delivery-slots	200 com server_now/timezone/items em alimentos; roupas retorna FEATURE_NOT_AVAILABLE
POST /internal/v1/reviews	web + dono; 201 somente após persistir snapshot
POST /internal/v1/orders	web + dono + chave UUIDv4; recebe store_id/review_id
GET /internal/v1/orders/{id}	Snapshot persistido; recurso de outro dono retorna 404
GET /internal/v1/order-attempts/{key}	Consulta sem criar nova tentativa ou reiniciar contadores

```
# services/orders/checkout_models.py
class OrderCommand(FrozenModel):
    store_id: Identifier
    review_id: UUID4 = Field(strict=False)

# services/orders/checkout_routes.py — excerto
async def get_order(order_id: UUID4, request: Request,
                    owner=Depends(trusted_session)):
    return await request.app.state.checkout.get_order(
        owner, order_id)
```

Antes da admissão, pedidos revalida configuração, validade da revisão, seleções, preços e janela. Mudança relevante produz REVIEW_CHANGED; não se aceita automaticamente um novo total. A chave de idempotência é única por loja e dono: mesmo comando recupera o resultado; conteúdo diferente gera 409. Cada revisão pode originar no máximo um pedido, por restrição local.

A reserva/admissão e o vínculo entre tentativa e pedido são locais e duráveis. Estados de tentativa: validating, accepted e rejected. Estados de pedido: payment_pending, payment_unknown, confirmed e payment_conflict. Nenhuma transação SQLite é mantida aberta durante HTTP. Consultas leem snapshots, mesmo após alteração do catálogo ou reinício.

6.3 Serviço de pagamentos e recuperação

Pagamentos implementa somente Pix simulado, sem desconto, QR Code ou dados bancários. O banco authorizations é privado e mantém unicidade por loja/pedido. O valor é recebido de pedidos já admitidos, nunca diretamente do navegador. O único estado de autorização simulada é approved; incerteza é um estado observado em pedidos, não uma segunda decisão financeira.

Rota interna	Permissão	Semântica
PUT /internal/v1/payments/by-order/{id}	orders	201 na primeira autorização; 200 no replay; 409 se valor/comando divergir
GET /internal/v1/payments/by-order/{id}	orders	200 na autorização encontrada; 404 explícito se ausente

```
# services/payments/routes.py — excerto com quebras visuais
@router.get('/by-order/{order_id}',
            response_model=AuthorizationResponse)
async def lookup(order_id: UUID4, request: Request):
    try:
        result = await asyncio.to_thread(
            request.app.state.repository.lookup, order_id)
    except StorageError:
        raise DomainError(
            'PAYMENT_STORAGE_UNAVAILABLE') from None
    if result is None:
        raise DomainError('PAYMENT_NOT_FOUND')
    return result
```

Sequência normal e falhas

A sequência é: persistir revisão; reservar chave; revalidar; admitir pedido em transação local; consultar autorização; somente diante de 404, autorizar com o mesmo order_id; persistir confirmação. Perda de resposta após aprovação é tratada por nova observação da autorização existente. A semântica de métodos e códigos HTTP é apoiada na RFC 9110 (Fielding; Nottingham; Reschke, 2022), mas a deduplicação é garantida pelas regras e restrições da aplicação.

O worker usa leases persistidos de quinze segundos, versão e gravação condicional. Há até cinco execuções por ciclo, com intervalos 2/4/8/16 segundos e retomada manual após cooldown de pelo menos trinta segundos. GET não reinicia contador. Reinício conserva estados e não cria outra autorização. Não se promete processamento exatamente uma vez nem transação distribuída.

6.3 Serviço de recomendações

GET /internal/v1/preferences e GET /internal/v1/recommendations consultam o perfil e as sugestões. POST /internal/v1/preferences salva escolhas; POST /internal/v1/recommendation-events registra view ou add_to_cart com UUID v4 idempotente; POST /internal/v1/recommendation-reset limpa e pausa o histórico. Todas exigem credencial exclusiva web e dono derivado da sessão. O gateway expõe os mesmos sufixos em /api/v1, com no-store e validação de origem nas mutações.

```
# services/recommendations/routes.py - excerto
@router.get('/recommendations',
            response_model=RecommendationResponse)
async def recommendations(request: Request,
                          owner=Depends(trusted_session)):
    return await request.app.state.recommendations.recommend(owner)
```

O componente puro ranking.py pontua categoria (+6), cor (+3), tamanho (+2) e categoria do histórico (até +6). Produtos já vistos recebem -2. Empates usam ID. O repositório privado impõe unicidade de eventos e limite de cem eventos por perfil. Sem transações abertas durante HTTP; respostas do catálogo são validadas e usam timeout e repetição limitada.

6.4 Portas e evidência dos serviços em execução

Processo	Porta local	Responsabilidade
Frontend Next.js	3000	Interface e intermediário de APIs
Catálogo FastAPI	8001	Consulta e resolução interna de seleções
Pedidos FastAPI	8002	Agenda, revisão, pedido e recuperação
Pagamentos FastAPI	8003	Autorização e consulta simuladas
Recomendações FastAPI	8004	Preferências, histórico e sugestões

As portas acima são definidas em tools/lps.py e Compose. Os testes de integração utilizam portas temporárias, o que permite executar serviços em processos independentes sem modificar recursos de demonstração. A aprovação da suíte comprova os cenários testados, mas não substitui a captura das portas exigida pelo item 6.4.

```
# PowerShell — serviços em terminais separados
.\.venv\Scripts\python.exe tools/lps.py service clothing `
  --service catalog
.\.venv\Scripts\python.exe tools/lps.py service clothing `
  --service payments
.\.venv\Scripts\python.exe tools/lps.py service clothing `
  --service orders
.\.venv\Scripts\python.exe tools/lps.py service clothing `
  --service recommendations

# Após iniciar o frontend, conferir as cinco portas:
Get-NetTCPConnection -State Listen |
  Where-Object { $_.LocalPort -in 3000,8001,8002,8003,8004 } |
  Select-Object LocalAddress,LocalPort,OwningProcess
```

Uma captura válida deve mostrar data/contexto da execução e processos/portas efetivamente ativos, ocultando credenciais. Também se deve registrar o estado agregado em http://127.0.0.1:3000/health/ready e distinguir liveness de readiness. Dependência indisponível degrada prontidão sem afirmar que o processo morreu.

Nota de evidência. Nesta execução foram capturados os quatro serviços e suas portas no painel da interface, todos prontos. Não houve captura de terminal com processos/PIDs. Não se apresenta tabela, comando ou saída fabricada como uma tela de processos em execução. O build do frontend permaneceu bloqueado por acesso negado no SWC; portanto, a evidência visual do item 6.4 precisa ser anexada em ambiente funcional.

Figura 7 - Serviços e portas da loja de roupas

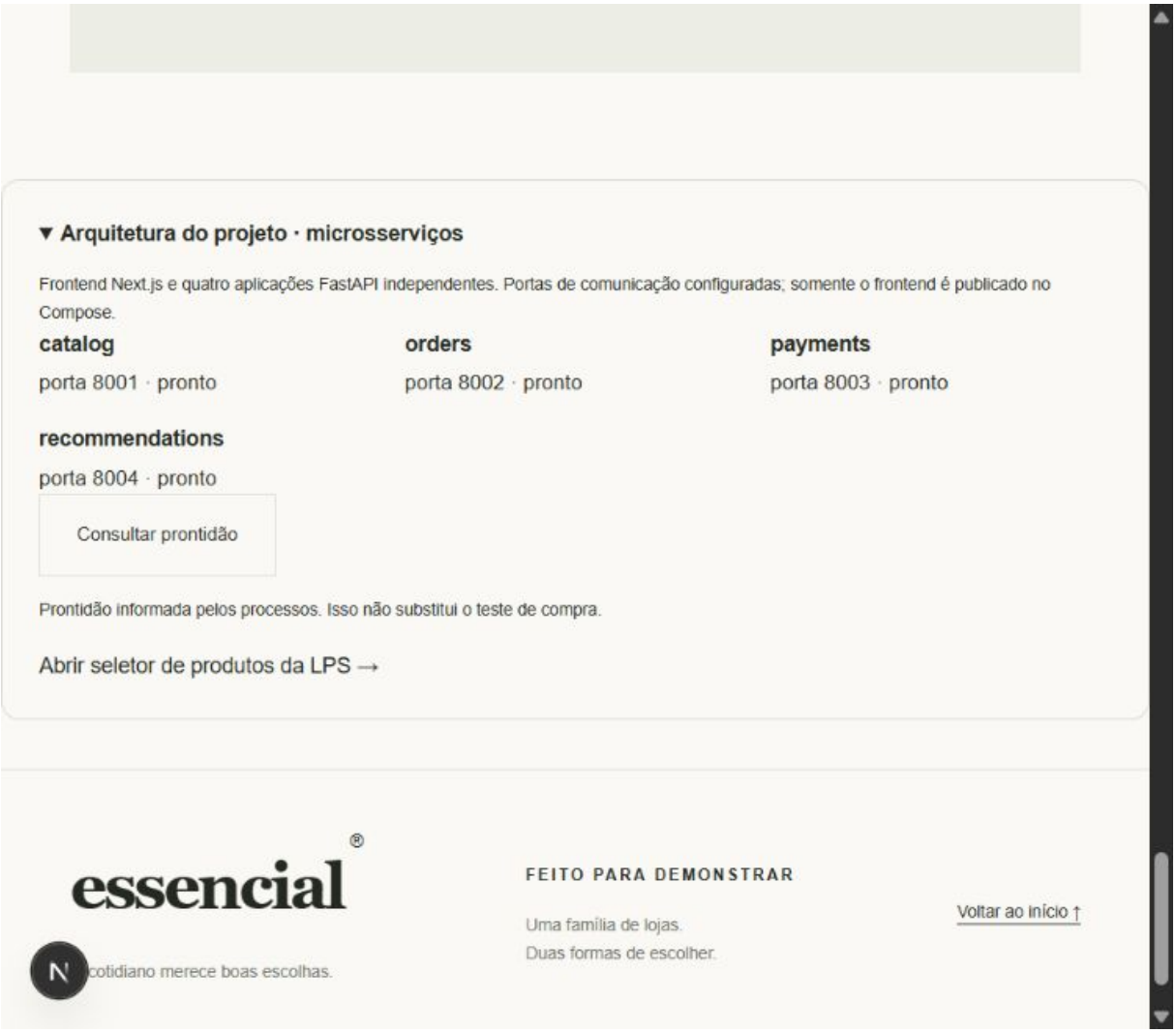


Figura 7 - Serviços e portas da loja de roupas. Fonte: captura real do sistema pela autora, assistida por ferramenta de automação (10 nov. 2026).

O painel da interface consultou a prontidão e apresentou catalog 8001, orders 8002, payments 8003 e recommendations 8004 como prontos. Trata-se da observação do painel, não de captura de terminal nem comprovação de isolamento ou reinício.

6.5 Construção de um produto a partir da LPS

A engenharia de aplicação instancia ativos comuns conforme a configuração. A derivação de roupas/alimentos já está representada pelos manifestos e pelo registro de componentes. Os passos abaixo constituem o procedimento executável documentado, sem confundir instrução de execução com execução aprovada.

Passo	Procedimento e verificação
1 — Preparar	Usar Python 3.11, Node/npm e dependências reais. Instalar requirements-dev.txt e npm ci pelo lockfile em ambiente com acesso aos índices.
2 — Configurar	Selecionar configurations/clothing.json ou food.json. Não incluir segredos no manifesto; bootstrap rejeita configuração inválida.
3 — Inicializar	Executar tools/lps.py init <variante>; preservar .local/<store_id>.env e SESSION_SECRET entre reinícios.
4 — Iniciar	Executar tools/lps.py run <variante> --dev ou iniciar os serviços por --service. Todos recebem a mesma identidade/hash.
5 — Conferir	Verificar prontidão, origem http://127.0.0.1:3000, catálogo correspondente e bancos privados por loja.
6 — Demonstrar	Roupas: combinações; alimentos: metadados e janela futura. Revisar e confirmar sem dados financeiros reais.
7 — Reiniciar	Conservar bancos/segredos/cookie; consultar pedido da sessão original. Não usar down -v no reinício normal.
8 — Trocar variante	Encerrar a implantação; executar a outra variante. Conferir isolamento de dados e sacola.

```
# Execução local na raiz do projeto
.\.venv\Scripts\python.exe tools/lps.py init clothing
.\.venv\Scripts\python.exe tools/lps.py run clothing --dev

# Alimentos, após Ctrl+C na execução anterior
.\.venv\Scripts\python.exe tools/lps.py init food
.\.venv\Scripts\python.exe tools/lps.py run food --dev
```

Um novo produto concreto pode ser derivado de uma configuração válida dentro do domínio suportado; a inclusão de um terceiro domínio demanda novos componentes e regras no schema, e não somente alteração de nome. O relatório demonstra a derivação documental das duas variantes existentes, sem afirmar que uma terceira aplicação foi construída ou testada.

7 Tecnologias utilizadas na implementação

Unidade	Tecnologias e linguagem	Dados e estado
Frontend e gateway	Next.js 15.3.1, React 19, TypeScript e Tailwind CSS 4; Node.js	Carrinho/referências no navegador; cookie assinado; sem banco próprio
Catálogo	Python 3.11, FastAPI 0.141.1, Pydantic 2.13.4 e Uvicorn 0.49.0	JSON fictício validado; snapshot imutável em memória
Pedidos	Python/FastAPI; Protocol, componentes e cliente HTTP tipado	SQLite privado; schema 1; revisões, tentativas e pedidos
Pagamentos	Python/FastAPI; repositório de autorização simulada	SQLite privado; authorizations e metadata
Recomendações	Python/FastAPI; ranking determinístico e cliente do catálogo	SQLite privado; profiles, events e metadata
Ambiente	Dockerfiles, Docker Compose e lançador Python/PowerShell	Volumes por serviço e projeto; arquivos de configuração somente leitura
Validação	unittest, Node test runner, vetores e testes HTTP	Recursos temporários; relatório local de resultados

Os números de versão são os fixados/registrados no projeto; React, TypeScript e Tailwind possuem faixas no package.json, com resolução no package-lock.json. O SQLite é acessado pela biblioteca padrão do Python, com transações curtas, chaves estrangeiras e inicialização compatível e repetível. O framework não transforma validação de schema em garantia de negócio: invariantes de revisão, pagamento e propriedade são implementadas pela aplicação (Python Software Foundation, documentação sqlite3; FastAPI, documentação oficial).

Validação e reprodutibilidade

Verificação	Evidência disponível
Backend completo	Histórico anterior: 67 testes aprovados. Incremento atual bloqueado pelo preflight de loopback
Offline atual	43 testes aprovados; não certificam HTTP/asyncio/worker
Fronteira web e configuração	16 testes Node e 24 vetores aprovados; sem comprovação de renderização
Checagem posterior do frontend	Lint e typecheck passaram; build falhou ao carregar next.config.ts por acesso negado no SWC
E2E visual, instalação limpa e Compose	Não aprovados; sem evidência de execução completa

Nota de evidência. A validação atual aprovou lint/typecheck e testes offline/web. Build falhou por acesso negado no SWC; HTTP completo/E2E foram bloqueados pelo preflight de loopback. pip check apontou conflito preexistente de setuptools/streamlit-vis-network; instalação limpa e Compose permanecem pendentes.

8 Funcionamento do produto e telas

A interface possui duas identidades visuais da mesma aplicação: “essencial” para roupas e “despensa” para alimentos. O layout usa tipografia de sistema, fotos ilustrativas locais e fallback SVG, busca, categorias, diálogo de produto, fluxo de compra e consulta. Não depende de fotografias remotas. A descrição abaixo está baseada no código de Storefront, ProductMedia e FoodInformation; não substitui a sequência de capturas reais exigida pelo roteiro.

Tela ou estado	Ação do usuário e comportamento	Microsserviço relacionado
Vitrine	Buscar/filtrar; consultar listagem real; backend indisponível mostra erro, sem catálogo de sucesso fictício	Catálogo, via Next.js
Detalhe de roupas	Selecionar combinação explícita habilitada e quantidade; desabilitada não representa estoque	Catálogo
Detalhe de alimentos	Ver ingredientes, alergênicos conhecidos/ausência declarada/desconhecidos e nutrição por porção	Catálogo
Sacola	Editar/remover linhas; persistência separada por loja; valores apenas estimados	Estado local; backend confere depois
Entrega	Informar endereço fictício; em alimentos escolher janela futura de São Paulo	Pedidos
Revisão	Receber snapshot e total do servidor; validade de dez minutos; sem reserva	Pedidos consulta catálogo
Confirmação	Enviar review_id e chave preservada; nenhum dado bancário/cartão	Pedidos e pagamentos simulados
Consulta	Ver estado, valores e entrega preservados; recuperar por tentativa com a sessão original	Pedidos; pagamento observado por pedidos

A sacola só é limpa após confirmação e se a seleção atual ainda corresponde à tentativa, preservando edições feitas enquanto a chamada estava em andamento. Estado incerto não significa compra fracassada. A interface acompanha automaticamente estados intermediários com até doze consultas, a cada dois segundos, sem reenviar mutações. O bootstrap de sessão é compartilhado entre recomendações e checkout para evitar cookies concorrentes. A interface permite consultar e reenviar a mesma chave, e bloqueia retomada com contexto de sessão diferente. Não há recuperação de pedido apenas por nome ou ID sem o cookie original.

Nota de evidência. As Figuras 8 a 16 apresentam execução real de roupas: vitrine, seleção, sacola, entrega, revisão, confirmação, consulta e recomendações. O pedido 6d252cef-57c0-4ba0-b5d9-cf4621427109 foi confirmado por R$ 89,90, com pagamento fictício. Foram usados nome e endereço fictícios. Alimentos, reinício, build de produção e Compose não foram validados nesta sessão.

Figura 8 - Vitrine da loja de roupas



Figura 8 - Vitrine da loja de roupas. Fonte: captura real do sistema pela autora, assistida por ferramenta de automação (10 nov. 2026).

A vitrine utiliza imagem local da camiseta e apresenta a identidade essencial. Os produtos e valores são fictícios; a captura registra a execução no navegador, não uma maquete.

Figura 9 - Combinações de tamanho e cor

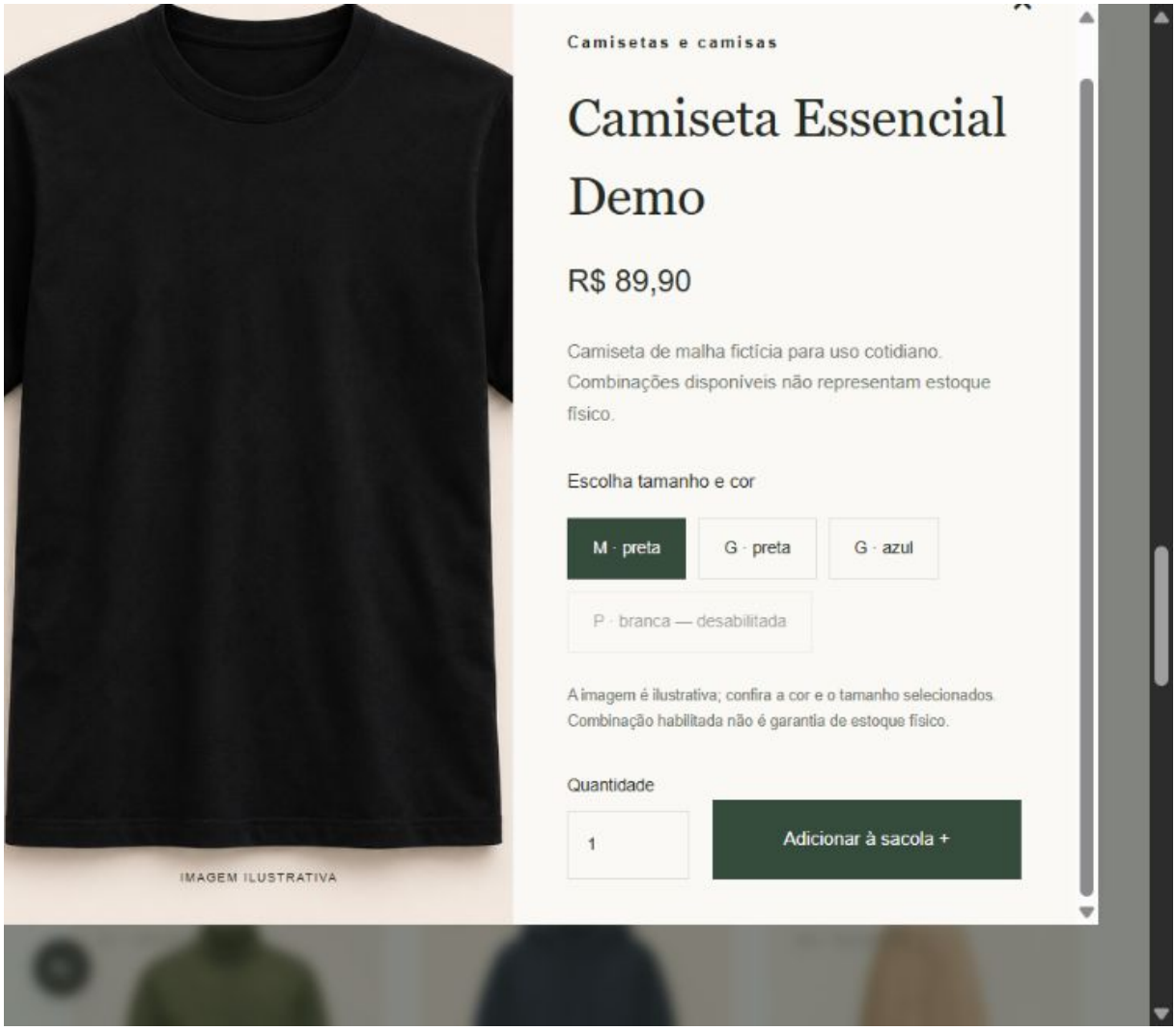


Figura 9 - Combinações de tamanho e cor. Fonte: captura real do sistema pela autora, assistida por ferramenta de automação (10 nov. 2026).

A camiseta foi selecionada em M e preta. As demais combinações aparecem explicitamente, incluindo a opção desabilitada. A imagem não constitui garantia de estoque físico.

Figura 10 - Sacola com uma seleção



Figura 10 - Sacola com uma seleção. Fonte: captura real do sistema pela autora, assistida por ferramenta de automação (10 nov. 2026).

A sacola contém uma camiseta M/preta, por R$ 89,90 estimados. A revisão subsequente confirma o valor no backend; o frontend não determina o preço final.

Figura 11 - Endereço fictício para entrega

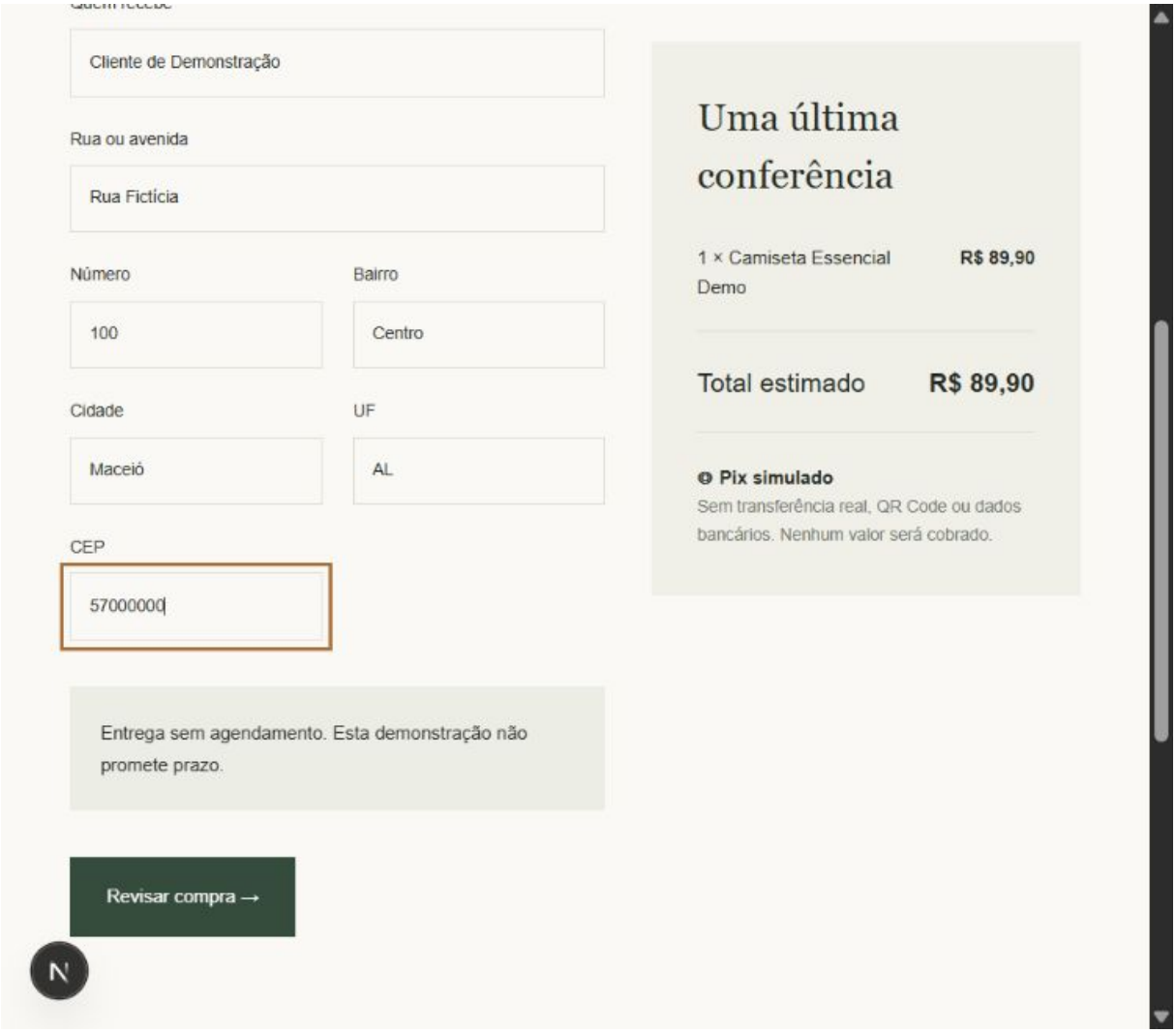


Figura 11 - Endereço fictício para entrega. Fonte: captura real do sistema pela autora, assistida por ferramenta de automação (10 nov. 2026).

O checkout sem cadastro recebeu Cliente de Demonstração, Rua Fictícia, 100, Centro, Maceió/AL e CEP 57000000. Roupas não exige agendamento nem promete prazo de entrega.

Figura 12 - Revisão da compra pelo servidor

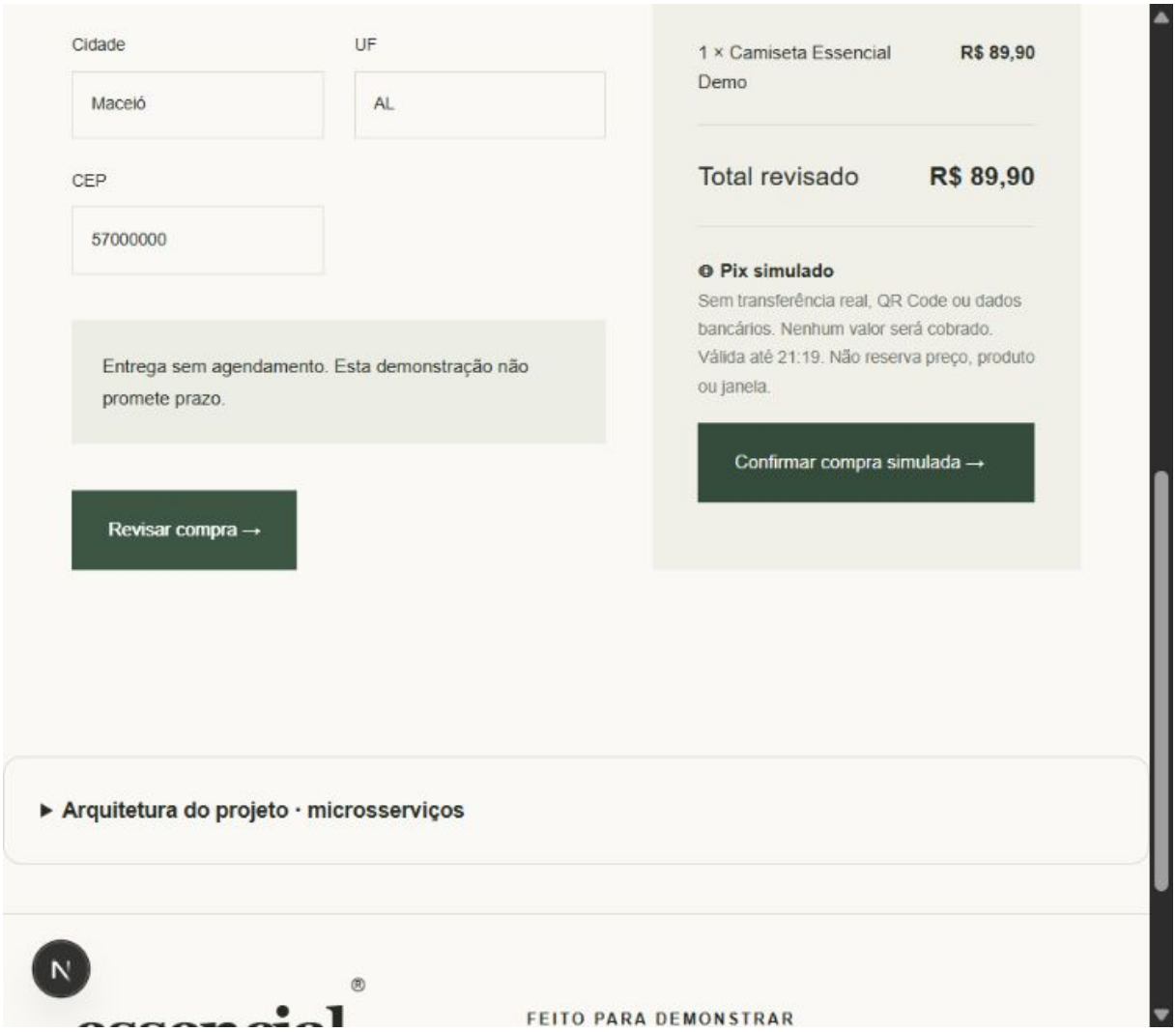


Figura 12 - Revisão da compra pelo servidor. Fonte: captura real do sistema pela autora, assistida por ferramenta de automação (10 nov. 2026).

O serviço de pedidos persistiu a revisão e retornou total de R$ 89,90, expiração e possibilidade de confirmar. O Pix é inteiramente simulado, sem transferência ou dados bancários.

Figura 13 - Confirmação da compra simulada



Figura 13 - Confirmação da compra simulada. Fonte: captura real do sistema pela autora, assistida por ferramenta de automação (10 nov. 2026).

A interface apresentou Pedido confirmado e Pagamento fictício aprovado. O identificador aleatório permite acompanhar o pedido na sessão original. Nenhum pagamento real foi realizado.

Figura 14 - Consulta posterior do pedido



Figura 14 - Consulta posterior do pedido. Fonte: captura real do sistema pela autora, assistida por ferramenta de automação (10 nov. 2026).

Após voltar à coleção e abrir Meu pedido, foi consultado novamente o mesmo identificador, com valor, combinação e endereço preservados. A navegação recarregou a página ao visitar o configurador; os processos de backend não foram reiniciados.

Figura 15 - Preferências de demonstração

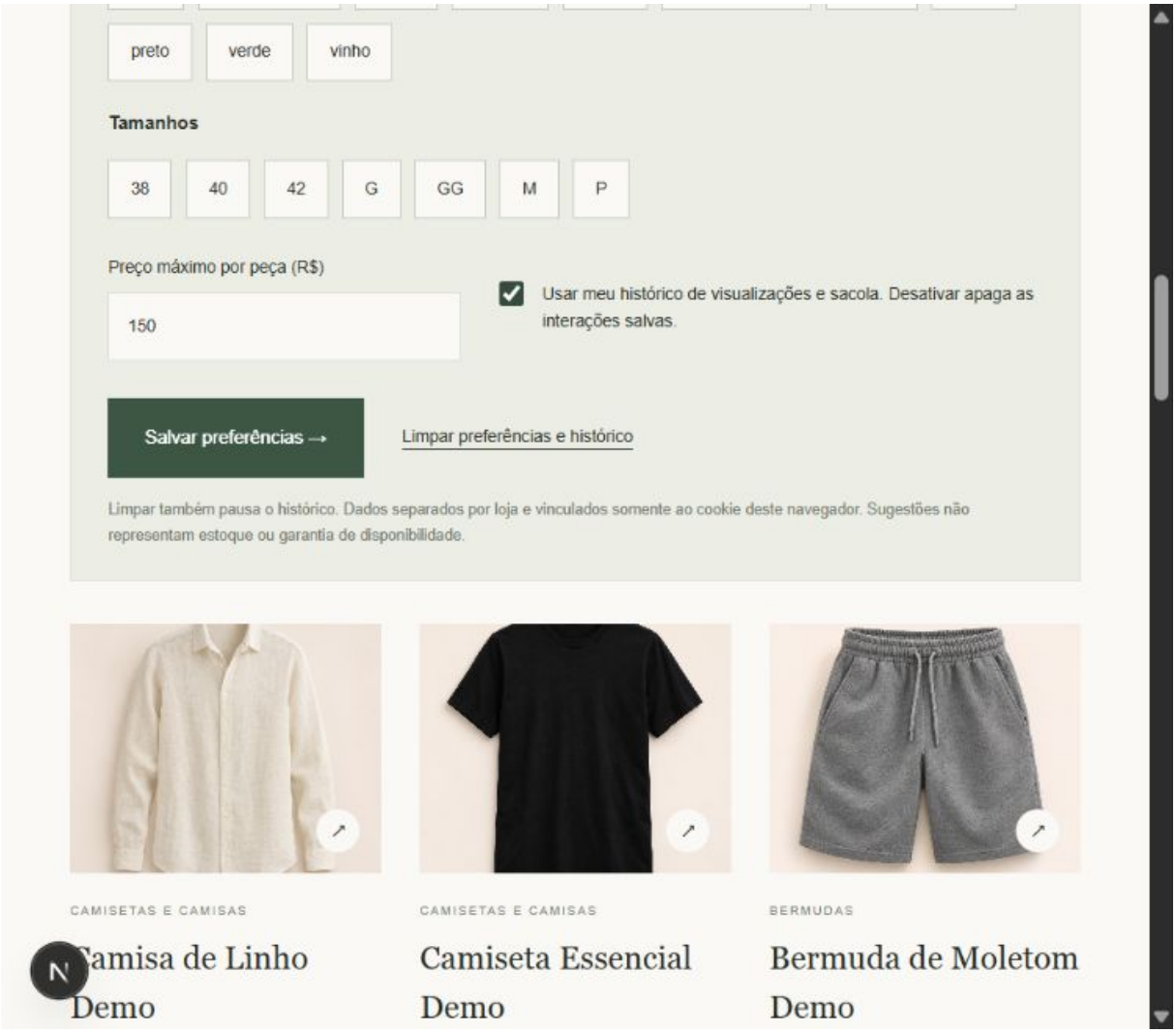


Figura 15 - Preferências de demonstração. Fonte: captura real do sistema pela autora, assistida por ferramenta de automação (10 nov. 2026).

Foram salvas a categoria Camisetas e camisas e o limite de R$ 150,00 por peça. O histórico desta sessão registrou duas interações: visualização e adição à sacola.

Figura 16 - Recomendações por preferências e histórico

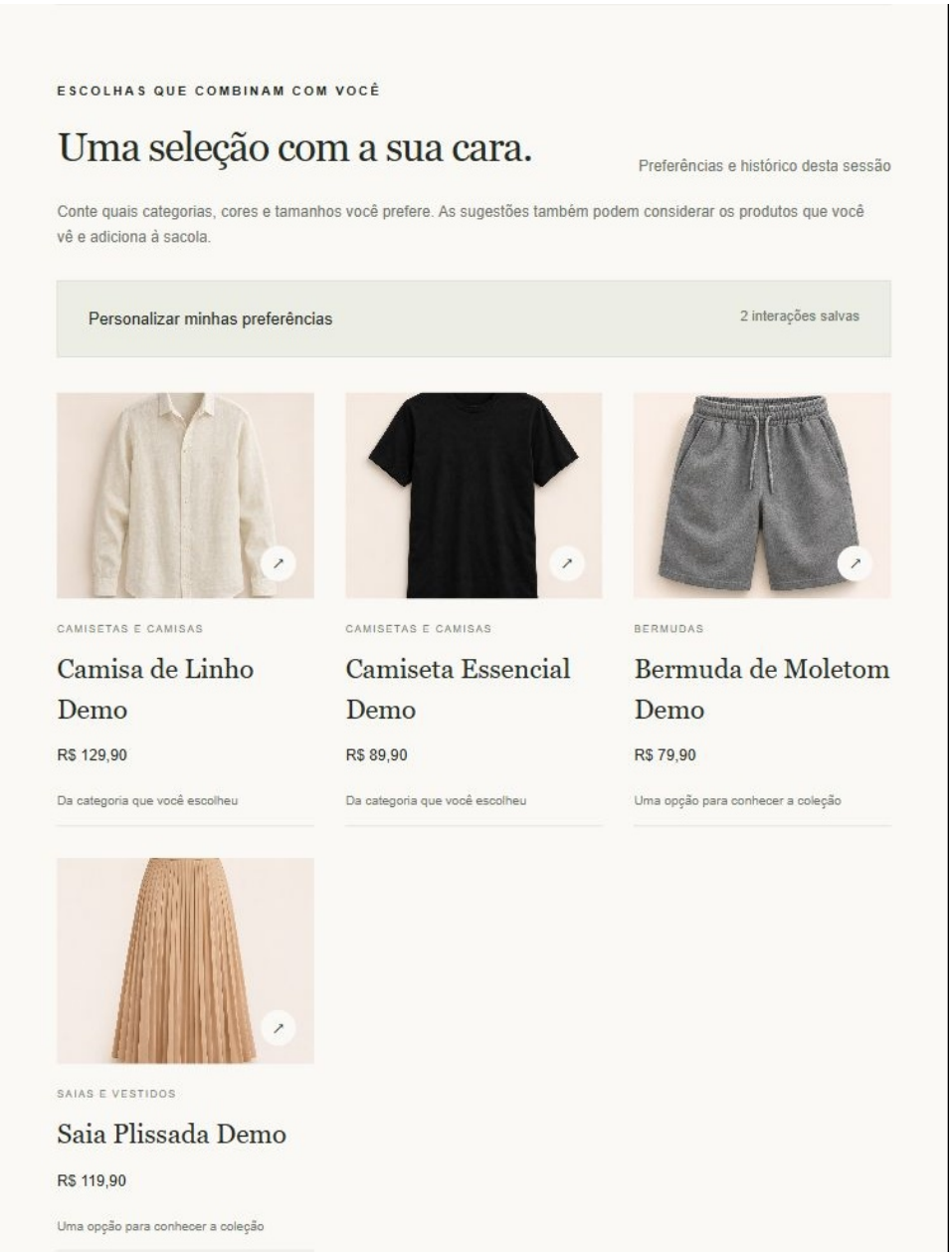


Figura 16 - Recomendações por preferências e histórico. Fonte: captura real do sistema pela autora, assistida por ferramenta de automação (10 nov. 2026).

As sugestões mostram motivos como categoria escolhida e categoria do histórico. Este recorte retangular da captura integral 12-pagina-completa.jpg preserva o conteúdo original, sem alterar textos ou resultados.

9 Código do projeto e disponibilização no GitHub

A autora informou o identificador público emilyrch. O endereço https://github.com/emilyrch corresponde ao perfil informado, não identifica o repositório do projeto e não comprova publicação ou versão disponível. Não foi encontrado um diretório .git na cópia analisada. Portanto, o URL específico solicitado na capa pelo item 9 permanece pendente.

Diretório ou arquivo	Conteúdo a disponibilizar
apps/store-ropa/src/	Frontend, sessão, gateway e componentes visuais
services/catalog/	APIs e aplicação de catálogo
services/orders/	Revisão, admissão, consultas, cliente de pagamento e worker
services/payments/	Autorização simulada durável
domain/ e foundation/	Contratos/componentes e infraestrutura comum
configurations/, shared/ e demo-data/	Manifestos, schemas/vetores e catálogos fictícios
tests/ e tools/	Validações, lançador e teste E2E preparado
README.md, docs/, Dockerfiles e Compose	Instalação, arquitetura, contratos, evidências e execução

O repositório deve excluir .local, bancos privados, segredos, node_modules, .venv e artefatos de build. O .gitignore contém essas exclusões, mas sua presença não comprova que uma publicação anterior esteja livre desses arquivos. Antes de publicar, devem ser revisados os arquivos efetivamente incluídos e o README com os passos reproduzíveis.

Limites da entrega verificada

O relatório não certifica funcionamento integral do código no GitHub: a publicação não foi demonstrada e a homologação do frontend não foi concluída. O backend possui evidências de testes completos, e as verificações posteriores de lint/typecheck não apresentaram erros. O build permanece impedido por permissão no ambiente. A execução manual de roupas registrada nas figuras é uma evidência específica; não aprova toda a suíte de integração nem a variante de alimentos.

Para reprodução e atualização das evidências, executar tools/validate.py em ambiente funcional e conservar o relatório gerado em .local/validation-latest.json como evidência local, sem publicar credenciais. O teste tools/e2e.py utiliza portas, bancos e tokens temporários; mesmo quando aprovado, ainda deve ser complementado por inspeção do navegador.

10 Referências bibliográficas

BECK, Kent et al. **Manifesto para Desenvolvimento Ágil de Software**. 2001. Disponível em: https://agilemanifesto.org/iso/ptbr/manifesto.html. Acesso em: 10 nov. 2026.

SCHWABER, Ken; SUTHERLAND, Jeff. **The Scrum Guide**. 2020. Disponível em: https://scrumguides.org/scrum-guide.html. Acesso em: 10 nov. 2026.

NORTHROP, Linda M. et al. **A Framework for Software Product Line Practice Version 5.0**. Pittsburgh: Software Engineering Institute, Carnegie Mellon University, 2012. Disponível em: https://www.sei.cmu.edu/library/a-framework-for-software-product-line-practice-version-50/. Acesso em: 10 nov. 2026.

OBJECT MANAGEMENT GROUP. **OMG Unified Modeling Language Version 2.5.1**. 2017. Disponível em: https://www.omg.org/spec/UML/2.5.1/. Acesso em: 10 nov. 2026.

FASTAPI. **Documentação oficial do FastAPI**. s.d.. Disponível em: https://fastapi.tiangolo.com/. Acesso em: 10 nov. 2026.

VERCEL. **Next.js 15 Route Handlers and Middleware**. s.d.. Disponível em: https://nextjs.org/docs/15/app/getting-started/route-handlers-and-middleware. Acesso em: 10 nov. 2026.

PYTHON SOFTWARE FOUNDATION. **sqlite3 DB API 2.0 interface for SQLite databases**. Documentação Python 3.11, s.d.. Disponível em: https://docs.python.org/3.11/library/sqlite3.html. Acesso em: 10 nov. 2026.

FIELDING, Roy; NOTTINGHAM, Mark; RESCHKE, Julian. **HTTP Semantics RFC 9110**. Internet Engineering Task Force, 2022. Disponível em: https://www.rfc-editor.org/rfc/rfc9110.html. Acesso em: 10 nov. 2026.

UNIVERSIDADE FEDERAL DE ALAGOAS. **Identidade Visual**. s.d.. Disponível em: https://ufal.br/ufal/comunicacao/manuais/identidade-visual. Acesso em: 10 nov. 2026.

UNIVERSIDADE FEDERAL DE ALAGOAS. **Elementos do Relatório do Projeto 2026 2**. Disciplina Tópico em Engenharia de Software, “Projetando Linhas de Produto de Software”. Material de orientação fornecido pela autora, 2 p., 2026. Nome do docente não informado.

ROCHA, Emily Vitória de Jesus. **Documentação técnica e código da LPS de comércio eletrônico**. Arquivos locais: docs/modelo-lps.md, arquitetura-atual.md, contratos-api.md, recuperacao.md, entrega-final.md e homologacao-2026-11-10.md; código de domain/, foundation/, services/ e apps/store-ropa/src/. Versão analisada em 10 nov. 2026. Repositório específico ainda não informado.